

— Web apps

Chapter 7: React routes

L. Espeel



— React Router

- React Router is een library die routeringsmogelijkheden biedt voor React-toepassingen.
- Mogelijkheden:
 - Meerdere pagina's creëren in een single-page applicatie
 - URL-parameters en queryreeksen verwerken
 - Browsergeschiedenis en navigatie beheren
 - Geneste routes en lay-outs maken
 - Beveiligde routes voor authenticatie implementeren

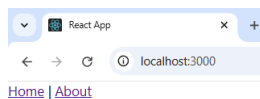
— Eenvoudige route (1)

- React Router zit niet standaard in de React library, dit dient apart geïnstalleerd te worden:
npm install react-router-dom
- Vervolgens dit toevoegen in code bovenaan waar we routing zullen toepassen (bv. in *App.js*):
`import { BrowserRouter, Routes, Route, Link } from "react-router-dom";`

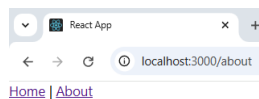
3

— Eenvoudige route (2)

- In bv. *App.js* kunnen we een navigatiestructuur opbouwen op basis van deze elementen:
 - **BrowserRouter**: root element om routing toe te laten.
 - **Link** of **NavLink**: navigatielink om de URL up te daten.
 - **Routes**: container om alle routedefinities bij te houden.
 - **Route**: definieert de mapping tussen een URL-pad en een component.
- Componenten *Home* en *About* stellen hier in feite *views* voor die ingeladen worden.



Home page



About page

```
function App() {  
  return (  
    <BrowserRouter>  
      <nav>  
        <Link to="/">Home</Link> {" | "  
        <Link to="/about">About</Link>  
      </nav>  
  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/about" element={<About />} />  
      </Routes>  
    </BrowserRouter>  
  );  
}  
  
export default App;
```

4

— Eenvoudige route (3)

Home.js:

```
function Home() {  
  return <h1>Home page</h1>;  
}
```

```
export default Home;
```

About.js:

```
function About() {  
  return <h1>About page</h1>;  
}
```

```
export default About;
```

5

— Eenvoudige route (4)

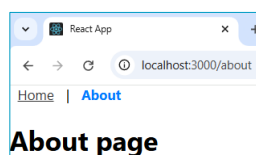
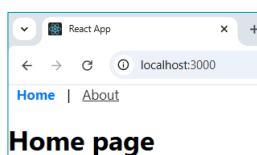
I.p.v. het *Link*-element, kan er ook gebruik gemaakt worden van het *NavLink*-element. Hiermee kunnen we styling toepassen als de link al dan niet actief is.

- Ideaal voor navigatiemenu's, breadcrumbs en tabbladen.
- Pas de import aan waarbij ook *NavLink* toegevoegd wordt:

```
import { <andere zaken ...>, NavLink } from "react-router-dom";
```

```
<BrowserRouter>  
  <nav>  
    <NavLink to="/" style={navLinkStyles}>Home</NavLink> {" | "}   
    <NavLink to="/about" style={navLinkStyles}>About</NavLink>  
  </nav>
```

```
const navLinkStyles = ({ isActive }) => ({  
  color: isActive ? "#007bff" : "#000333",  
  textDecoration: isActive ? "none" : "underline",  
  fontWeight: isActive ? "bold" : "normal",  
  padding: "5px 10px"  
});
```



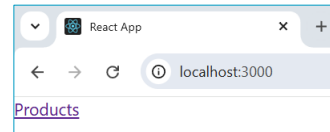
6

— Geneste routes (1)

Dit is een *Route*-element binnenin een ander *Route*-element.
Binnenin het *Products*-component zullen de componenten *CarProducts* en *BikeProducts* ingeladen worden.

```
<BrowserRouter>
  <nav>
    <Link to="/products">Products</Link>
  </nav>

  <Routes>
    <Route path="/products" element={<Products />}>
      <Route path="car" element={<CarProducts /> } />
      <Route path="bike" element={<BikeProducts /> } />
    </Route>
  </Routes>
</BrowserRouter>
```



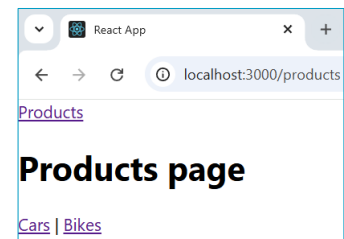
7

— Geneste routes (2)

```
Products.js: import { Link, Outlet } from "react-router-dom";

function Products() {
  return (
    <div>
      <h1>Products page</h1>
      <nav style={{ marginBottom: "20px" }}>
        <Link to="/products/car">Cars</Link> { " | " }
        <Link to="/products/bike">Bikes</Link>
      </nav>
      <Outlet />
    </div>
  );
}

//...
```



Het *Outlet*-element specificeert waar de inhoud van de child (*CarProducts* of *BikeProducts*) moet ingeladen worden.

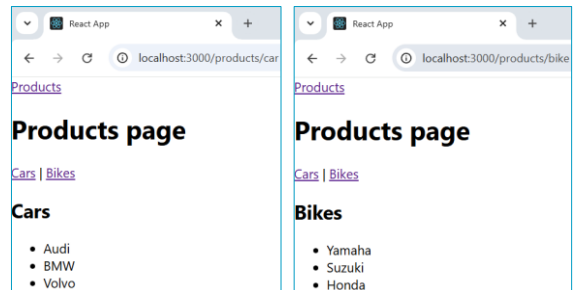
8

— Geneste routes (3)

Child-componenten:

```
function CarProducts() {  
  return (  
    <div>  
      <h2>Cars</h2>  
      <ul>  
        <li>Audi</li>  
        <li>BMW</li>  
        <li>Volvo</li>  
      </ul>  
    </div>  
  );  
}  
  
//...
```

```
function BikeProducts() {  
  return (  
    <div>  
      <h2>Bikes</h2>  
      <ul>  
        <li>Yamaha</li>  
        <li>Suzuki</li>  
        <li>Honda</li>  
      </ul>  
    </div>  
  );  
}  
  
//...
```



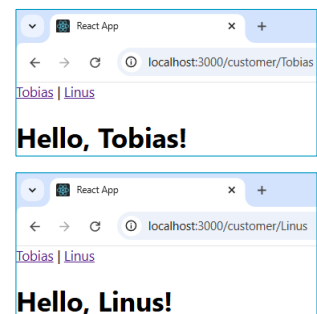
9

— URL-parameters (1)

We gebruiken variabelen om toe te voegen aan routepaden, dit kan gebruikt worden om data door te geven aan de verschillende componenten.

Bv. <http://localhost:3000/customer/Tobias> → de URL-parameter is hier Tobias.

```
<BrowserRouter>  
  <nav>  
    <Link to="/customer/Tobias">Tobias</Link> { " | " }  
    <Link to="/customer/Linus">Linus</Link>  
  </nav>  
  
  <Routes>  
    <Route path="/customer/:firstname" element={<Info />} />  
  </Routes>  
</BrowserRouter>
```



10

— URL-parameters (2)

In het *Info*-component maken we gebruik van de *useParams()*-hook om de URL-parameter op te kunnen vragen:

```
import { useParams } from "react-router-dom";

function Info() {
  const { firstname } = useParams();
  return <h1>Hello, {firstname}!</h1>;
}

export default Info;
```

11

— URL-parameters (3)

Alternatieven om data door te geven aan andere componenten:

- Via **query-parameter** (*?key1=value1&key2=value2&...*)

```
<BrowserRouter>
  <nav>
    <Link to="/customer?firstname=Peter">Peter</Link>
  </nav>

  <Routes>
    <Route path="/customer" element={<Info />} />
  </Routes>
</BrowserRouter>

import { useSearchParams } from "react-router-dom";

function Info() {
  const [searchParams, setSearchParams] = useSearchParams();
  const firstname = searchParams.get("firstname");
  return <h1>Hello, {firstname}!</h1>;
}

export default Info;
```

- Via **attributen** van componenten en die vervolgens via *props* uitlezen (zie o.a. hoofdstuk 1 p. 44).
- Via *useContext()*-hook (zie hoofdstuk 4 p. 130).

12

— React Router V7

Vanaf React Router V7 kan je naast voorgaande mogelijkheden met elementen *BrowserRouter*, *Link/NavLink*, *Routes* en *Route*, ook gebruik maken van de functie *createBrowserRouter* om de paden op te bouwen en te mappen met de componenten.

```
1 import { createBrowserRouter } from "react-router";
2
3 function Root() {
4   return <h1>Hello world</h1>;
5 }
6
7 const router = createBrowserRouter([
8   { path: "/", Component: Root },
9 ]);
```

Meer info: <https://reactrouter.com/start/data/installation>

13

— Frontend en backend (1)

Vaak wil je in de views gegevens dynamisch inladen, al dan niet afkomstig uit een databank. Om dit mogelijk te maken, kun je twee projecten opzetten:

1. Frontend-project met React

De componenten bevatten *fetch*-functies (geëncapsuleerd in de *useEffect*-hook, zie hoofdstuk 4) om data op te halen van de backend via een URL met een poortnummer.

- Gebruik *localhost* als host indien beide projecten op dezelfde machine draaien.
- Het uitwisselingsformaat tussen frontend en backend is vrij te kiezen; vaak wordt JSON gebruikt.

14

— Frontend en backend (2)

2. Backend-project met bv. Node.js/Express

Dit project zet een webserver op die luistert naar een bepaald poortnummer waarop de frontend zal verbinden.

- De backend heeft ook de mogelijkheid om te verbinden met een databank.

Een voorbeeld van zo'n opzet vind je op <https://www.angularminds.com/blog/how-to-create-a-react-app-with-a-node-backend>.

15

— Frontend en backend (3)

Een alternatief is gebruik maken van **Next.js**, een open-source React-framework dat bovenop React gebouwd is. Next.js voegt daar een server-side laag en projectstructuur aan toe.

Meer info op:

- <https://nextjs.org>
- <https://www.geeksforgeeks.org/nextjs/nextjs-tutorial>

16